

## Assignment 4

In this task, you will need to finish the full pipeline for the project from assignment 1.

Your pipeline should include the following: gather the full pipeline of the model on the sagemaker, databricks or Kubeflow, and combine with the previous tasks using you favourite tools for everything (or tools that make the most sense) and prepare a short presentation/demo.

The pipeline should include the following steps: data gathering (using airflow or other orchestrator, from prev. assignment), data processing step, model training step, parameters tuning step (this one is optional, if it makes sense), registering the model in model registry, logging all needed metrics, evaluating to detect if it is better then previous model, serving the best model.

So what's different, now it all runs in Databricks as well.

### GitHub Issue Label Assistant: Final Pipeline

- **Data Ingestion** (src/[ingestion.py](#)). Source: GitHub REST API (pytorch/pytorch issues, ~20k most recently created). Paginated, rate-limit aware (respects x-ratelimit-remaining/reset headers)

Output: raw JSON

- **Preprocessing** (src/[preprocessing.py](#)). Filters out pull requests and bot-authored issues. Cleans text: strips code blocks, URLs, markdown. Excludes non-content workflow labels . Keeps the top-N most frequent labels

Output: cleaned, labeled training set

- **Model Training** — two variants, trained in parallel: **Baseline** (train\_baseline.py): word-level TF-IDF + One-vs-Rest Logistic Regression, **Champion candidate** (train\_champion.py): word + character-level TF-IDF (FeatureUnion) + One-vs-Rest Logistic Regression — a real architectural improvement, not a hyperparameter tweak. Chronological (time-based) train/holdout split, not random — mimics production and exposes drift
- **Experiment Tracking & Model Registry** — MLflow ([evaluate.py](#)). Every run logs params, metrics (micro-F1, macro-F1, precision@3, recall@3), and the model artifact (with signature). Two interchangeable backends, same code, env-var toggle:

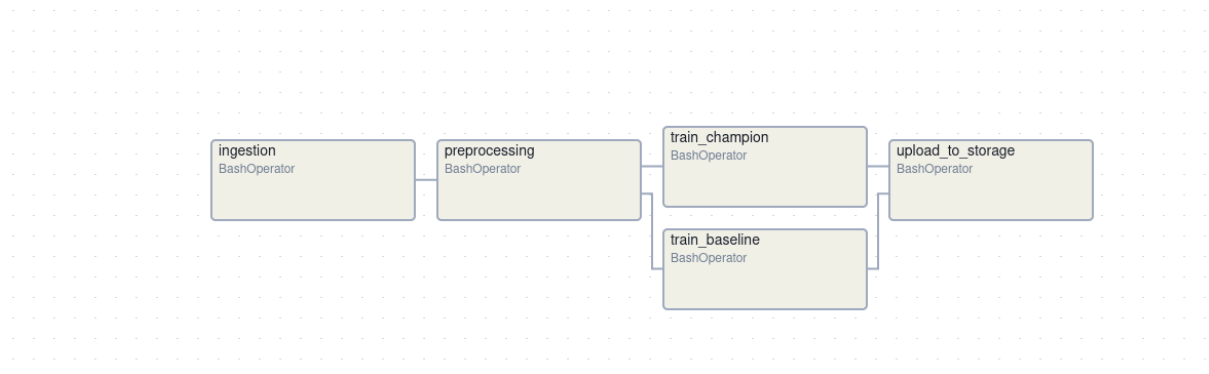
**Local:** self-hosted MLflow server + MinIO (S3-compatible) artifact store

**Managed platform (Databricks):** Databricks-hosted MLflow + Unity Catalog Model Registry

- **Champion/Challenger Promotion** (automatic). Every new model version is registered and tagged challenger. Compared against the current champion on micro-F1. Promoted to champion if it beats the incumbent by a defined margin — otherwise stays challenger
- **Orchestration** — Apache Airflow. Daily DAG: ingestion → preprocessing → [train\_baseline, train\_champion]. Runs in Docker, the same DAG/scripts can target either MLflow backend with no code changes.
- **Serving** — FastAPI. Loads both champion and challenger directly from the registry at startup. Endpoints: POST /predict/champion, POST /predict/challenger, POST /predict (champion alias), GET /health

- **Infrastructure** — Docker Compose. Core services: MinIO, MLflow, Airflow, FastAPI  
docker-compose.databricks.yml overrides training/serving env vars to point at the Databricks-hosted backend, leaving the base pipeline untouched

Airflow:



It run evaluation during training, forgot to get it out, but for now won't touch stuff that works

Our models in Databricks MLFlow:

| Run Name        | Created        | Dataset | Duration | Source      | Models                      |
|-----------------|----------------|---------|----------|-------------|-----------------------------|
| update-ek-456   | 3 minutes ago  | -       | 2.9min   | train_ba... | workspace.naysayer.issue... |
| knofly-fly-483  | 21 minutes ago | -       | 4.6min   | train_ch... | workspace.naysayer.issue... |
| steady-show-906 | 25 minutes ago | -       | 2.8min   | train_ba... | workspace.naysayer.issue... |

Champion:

```

Curl
curl -X POST \
  http://localhost:8000/predict/champion \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "A bug",
    "body": "this bug has a tedious resemblance to the environmental shift"
  }'

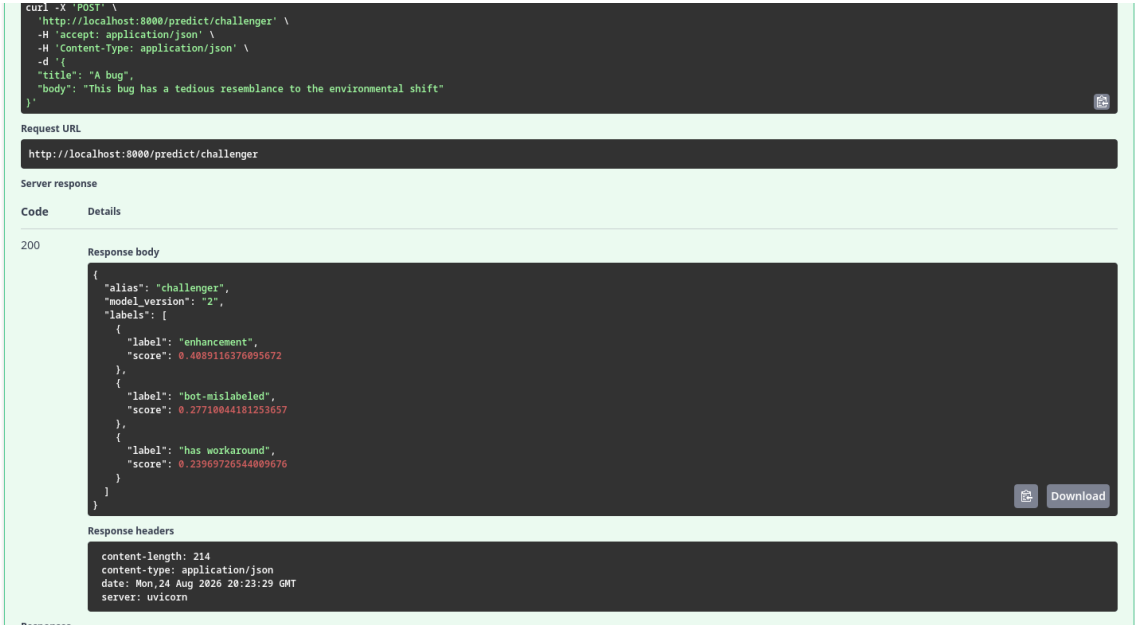
Request URL
http://localhost:8000/predict/champion

Server response
200

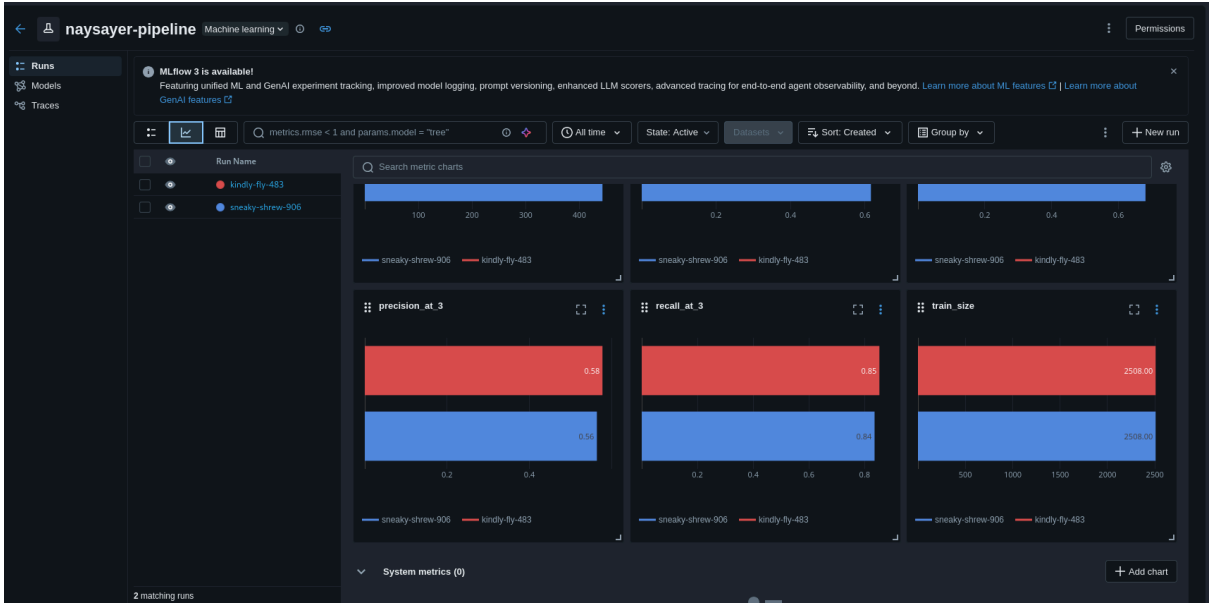
Response body
{
  "label": "champion",
  "model_version": "2",
  "labels": [
    {
      "label": "enhancement",
      "score": 0.480312370695972
    },
    {
      "label": "not-mislabelled",
      "score": 0.2771884181253607
    },
    {
      "label": "has-mislabelled",
      "score": 0.2395972554489676
    }
  ]
}

Response headers
content-length: 212
content-type: application/json
date: Mon, 24 Aug 2020 20:15:42 GMT
server: uvicorn
  
```

Challenger:



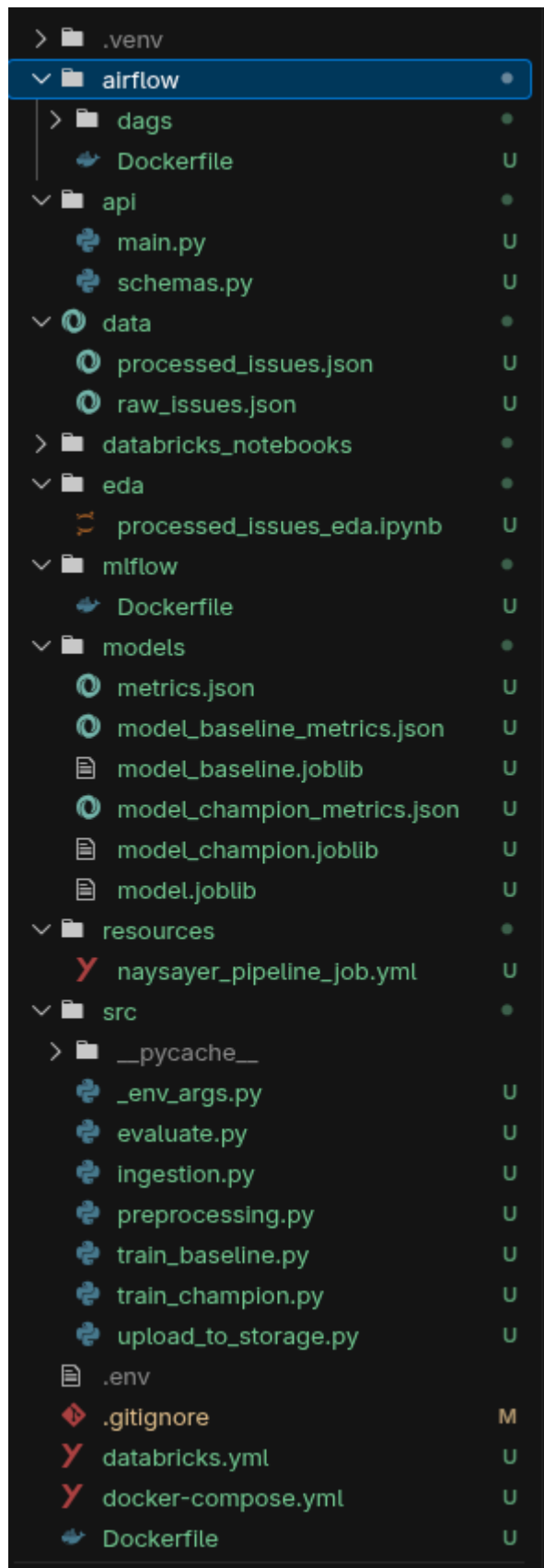
Model metrics:



Containerization:

|              |                 |                          |                |                         |
|--------------|-----------------|--------------------------|----------------|-------------------------|
| 18f57e7dba5b | naysayer-api    | "uvicorn api.main:ap..." | 13 minutes ago | Up 5 minutes            |
| 26433fd1d4de | naysayer-mlflow | "mlflow server --hos..." | 13 minutes ago | Up 13 minutes (healthy) |

Github: <https://github.com/GrimLord16/naysayer>



The file structure of the project